DIVYA R WAGHMODE

A-28

PDS PRACTICAL 4

Aim - Numerical Computation with NumPy: Perform different computations using NumPy library.

1 . Explain the role of NumPy in the Python scientific computing ecosystem.

Role of NumPy in the Python Scientific Computing Ecosystem

NumPy (Numerical Python) is the core library for numerical and scientific computing in Python.

Key roles: Provides a powerful N-dimensional array object (ndarray) Acts as a foundation for other libraries like Pandas, Matplotlib, and Scikit-learn Supports fast mathematical operations on large datasets Helps in working with: Linear algebra Statistics Random number generation

2.Why is NumPy Faster than Pure Python?

NumPy is faster than regular Python because:

a. Uses compiled code (C language) NumPy operations are written in C, not pure Python This makes execution much faster

b. Vectorization Works on entire arrays at once instead of loops Example:

for i in range(len(a)):

```
c[i] = a[i] + b[i]
```

c = a + b

c. Less memory usage Arrays store data more efficiently than Python lists

d. Better CPU usage Uses optimized algorithms and memory handling

3 . What types of mathematical operations can be performed using NumPy?

Types of Mathematical Operations in NumPy

NumPy supports a wide range of operations:

a. Basic arithmetic

Addition, subtraction, multiplication, division

a + b, a - b, a * b, a / b

b. Statistical operations

Mean, median, standard deviation np.mean(a), np.std(a)

c. Linear algebra

Matrix multiplication, determinant, inverse

np.dot(a, b)

d. Trigonometric functions

sin, cos, tan

np.sin(a), np.cos(a)

e. Exponential and logarithmic

np.exp(a), np.log(a)

f. Random number generation

np.random.rand()

4.Explain indexing and slicing in NumPy arrays. Indexing and Slicing in NumPy Arrays

Indexing

Access specific elements using position:

a[0]
a[2]

2D Indexing

a[1, 2]

Slicing

Extract a range of elements:

a[1:4]

a[::2]

2D slicing

a[0:2, 1:3]

Indexing = get one value

Slicing = get multiple values

5.How does NumPy integrate with libraries like:

a. Pandas

b. Matplotlib

c. Scikit-learn

8 Integration of NumPy with Other Libraries

a. Pandas

Pandas uses NumPy internally DataFrames are built on NumPy arrays

import pandas as pd

df = pd.DataFrame(np.array([[1,2],[3,4]]))

b. Matplotlib

Used for plotting graphs

NumPy generates numerical data for plots

import matplotlib.pyplot as plt

x = np.arange(0, 10)

y = x**2

plt.plot(x, y)

c. Scikit-learn

Machine learning library

Uses NumPy arrays for input data

from sklearn.linear_model import LinearRegression

model.fit(X, y)

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])

arr2 = np.array([[1, 2, 3], [4, 5, 6]])

zeros = np.zeros((2, 3))
ones = np.ones((2, 2))
identity = np.eye(3)

print(arr1)
print(arr2)
print(zeros)
print(ones)
print(identity)
```

```
[1 2 3 4]
[[1 2 3]
 [4 5 6]]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
```

```
 [1. 1.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

```
Addition: [11 22 33]
Subtraction: [ 9 18 27]
Multiplication: [10 40 90]
Division: [10. 10. 10.]
```

```
data = np.array([10, 20, 30, 40, 50])

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
print("Sum:", np.sum(data))
print("Min:", np.min(data))
print("Max:", np.max(data))
```

```
Mean: 30.0
Median: 30.0
Standard Deviation: 14.142135623730951
Sum: 150
Min: 10
Max: 50
```

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])


print("Matrix Addition:\n", A + B)

print("Matrix Multiplication:\n", np.dot(A, B))

print("Transpose of A:\n", A.T)
```

```
Matrix Addition:
 [[ 6  8]
 [10 12]]
Matrix Multiplication:
 [[19 22]
 [43 50]]
Transpose of A:
 [[1 3]
 [2 4]]
```

```
arr = np.array([10, 20, 30, 40, 50])

print("First element:", arr[0])
print("Last element:", arr[-1])
print("Slice (1 to 3):", arr[1:4])


arr2d = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Second row:", arr2d[1])
print("Element (2,2):", arr2d[1,1])
```

```
First element: 10
Last element: 50
Slice (1 to 3): [20 30 40]
Second row: [4 5 6]
Element (2,2): 5
```

```
arr = np.arange(1, 10)

reshaped = arr.reshape(3, 3)
print("Original:", arr)
print("Reshaped:\n", reshaped)
```

```
Original: [1 2 3 4 5 6 7 8 9]
Reshaped:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```


rand = np.random.rand(3)
rand_int = np.random.randint(1, 10, size=5)

print("Random floats:", rand)
print("Random integers:", rand_int)
```

```
Random floats: [0.77766955 0.90662848 0.65247381]
Random integers: [4 7 5 7 6]
```

```
arr = np.array([10, 25, 30, 45, 50])

filtered = arr[arr > 30]
print("Values greater than 30:", filtered)
```

```
Values greater than 30: [45 50]
```

```
arr = np.array([1, 4, 9, 16])

print("Square root:", np.sqrt(arr))
print("Exponential:", np.exp(arr))
print("Log:", np.log(arr))
```

```
Square root: [1. 2. 3. 4.]
Exponential: [2.71828183e+00 5.45981500e+01 8.10308393e+03 8.88611052e+06]
```

```
Log: [0.         1.38629436 2.19722458 2.77258872]
```

Conclusion :

NumPy is a powerful library used for numerical computation in Python. In this practical, we performed various operations such as array creation, mathematical calculations, statistical analysis, matrix operations, indexing, slicing, and reshaping of data. We also explored advanced features like broadcasting and random number generation.

These operations showed that NumPy makes complex calculations faster and more efficient compared to traditional Python methods. It also helps in handling large datasets easily, which is very useful in data science and scientific computing.

Overall, NumPy provides a strong foundation for numerical analysis and is an essential tool for performing efficient and accurate computations in Python.